

MODULE 7 · MULTI-TENANCY & SECURITY

ServiceAccounts

Pod identity — how a Pod proves who it is to the API

Pods call the API too — they need an identity

You already authenticate to the API server as a **human user** — a cert, a token, an OIDC login. But plenty of API calls come from inside the cluster: a Pod listing other Pods, a controller watching a resource.

Kubernetes has no "User" API object for that. Instead every Pod runs as a **ServiceAccount** — a first-class, namespaced identity built for machines, not people.

BY THE END YOU CAN

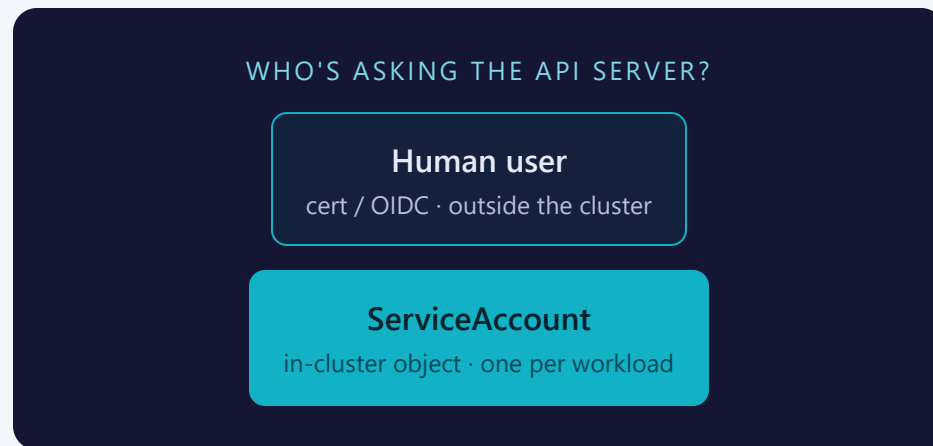
- Explain ServiceAccount vs human user
- Find the token Kubernetes mounts into a Pod
- Use that token to call the API
- Bind RBAC permissions to a ServiceAccount
- Explain the default SA and opt-out

CONCEPT

Two kinds of identity, one API server

- **Human users** are managed outside the cluster — certs, OIDC, an identity provider. There's no `User` object to `kubectl get`.
- **ServiceAccounts** are real, namespaced API objects — created, listed, and RBAC-bound like anything else.
- Every namespace gets a default `ServiceAccount` automatically; a `Pod` uses it unless you set `serviceAccountName`.

Same authentication pipeline, same RBAC layer — just a different kind of subject.



Kubernetes mounts the credential for you

FILE IN THE MOUNTED VOLUME	PURPOSE
token	JWT — the Bearer credential the Pod presents to the API server
ca.crt	Cluster CA certificate, used to verify the API server's identity
namespace	The namespace this ServiceAccount belongs to

MENTAL MODEL

No manifest field required: the kubelet projects this volume at `/var/run/secrets/kubernetes.io/serviceaccount/` into every Pod automatically, unless you opt out.

Create the identity, then bind the Pod to it

```
# serviceaccount.yaml
apiVersion: v1
kind: ServiceAccount
metadata:
  name: api-explorer
  namespace: training
```

```
# pod.yaml
apiVersion: v1
kind: Pod
metadata:
  name: api-explorer-pod
  namespace: training
spec:
  serviceAccountName: api-explorer # binds Pod → SA
  containers:
    - name: curl
      image: curlimages/curl:latest
      command:
        - sleep
        - "3600"
```

Create the ServiceAccount **before** the Pod references it — `spec.serviceAccountName` is the only Pod field involved, both must share a namespace, and a Pod with no name set falls back to default.

Imperative to explore, declarative to keep

IMPERATIVE — FAST, ONE-OFF

```
kubectl create serviceaccount api-explorer -n training
```

Creates the object now, no file to review

DECLARATIVE — VERSIONABLE, REPEATABLE

```
kubectl apply -f serviceaccount.yaml
```

The SA, its RoleBinding, and the Pod all live in Git together

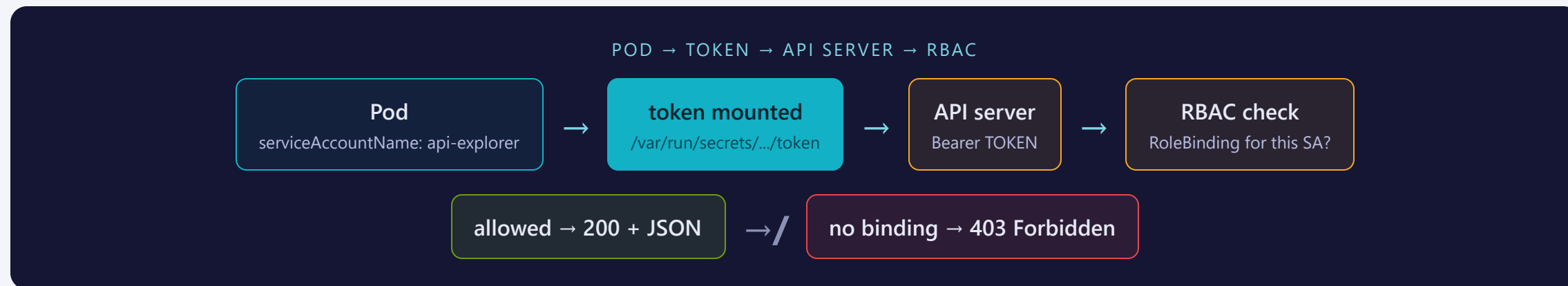
THE BRIDGE TRICK

Generate YAML from an imperative command, then edit & keep it:

```
$ kubectl create serviceaccount api-explorer -n training \
  --dry-run=client -o yaml > serviceaccount.yaml
```

SEE IT

One request, gated end to end



Skip the Pod entirely to test this: `kubectl auth can-i list pods --as=system:serviceaccount:training:api-explorer`.

A ServiceAccount is a name until you bind it

rolebinding.yaml

```
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: api-explorer-binding
  namespace: training
subjects:
- kind: ServiceAccount
  name: api-explorer
  namespace: training
roleRef:
  kind: Role
  name: api-explorer-role
  apiGroup: rbac.authorization.k8s.io
```

Creating a ServiceAccount grants **zero** permissions — access comes from a RoleBinding whose subject targets it.

Not every RBAC subject is a ServiceAccount

SUBJECT KIND	TYPICAL TARGET
User	Human, authenticated externally (cert / OIDC)
Group	A set of users
ServiceAccount	A Pod's in-cluster identity

Every namespace also gets a default SA — pods that skip `serviceAccountName` use it, and by design it usually has no extra permissions bound.

MENTAL MODEL

Identity and authorization are separate steps: the token proves *who*, RBAC decides *what*. No binding means the SA can authenticate but do nothing.

Where people trip

- **Forgetting serviceAccountName.** The Pod silently runs as default, which usually has no RBAC bound — API calls fail with 403.
- **Debugging only from inside a Pod.** `kubectl auth can-i --as=system:serviceaccount:NS:SA` checks the binding instantly, no Pod needed.
- **Leaving tokens mounted unnecessarily.** A Pod with no API needs but `automountServiceAccountToken: true` widens the blast radius if compromised.
- **Wrong subject kind in a binding.** RBAC subjects match exactly — `kind: ServiceAccount` with the right namespace, or the binding silently does nothing.

Commands to keep at hand

```
$ kubectl create serviceaccount NAME -n NS      # create imperatively
$ kubectl get sa -n NS                        # list ServiceAccounts
$ kubectl get pod NAME -o jsonpath='{.spec.serviceAccountName}'
$ kubectl exec NAME -- cat /var/run/secrets/kubernetes.io/serviceaccount/token
$ kubectl auth can-i VERB RESOURCE -n NS \
    --as=system:serviceaccount:NS:SA          # test without a Pod
$ kubectl create rolebinding NAME --role=ROLE \
    --serviceaccount=NS:SA -n NS              # bind RBAC to a SA
```

Tip: `--as=system:serviceaccount:NS:SA` impersonates the SA for any `kubectl auth can-i` check.

RECAP

ServiceAccounts in one breath

- A ServiceAccount is identity for a Pod, not a person
- The kubelet auto-mounts a token, `ca.crt`, and namespace
- The token alone grants nothing — RBAC bindings do
- default SA exists per namespace; opt out with `automountServiceAccountToken: false`

HANDS-ON

Now run Lab 7.3
[labs/7.3/](#)

Next: [7.4 — Kubeconfig & Contexts](#)